

Documento Técnico: Pruebas y Validaciones Realizadas — Proyecto HeartCoin

Introducción

Este documento consolida la evidencia de las pruebas y validaciones efectivamente realizadas sobre el proyecto HeartCoin, cubriendo pruebas funcionales de backend y frontend, pruebas de integración, validación de flujos transaccionales, validación de saldos y movimientos, pruebas de seguridad básica de APIs, y validación de errores y casos borde. Para cada categoría se documenta **con precisión el método real utilizado** —ejecución en vivo contra el entorno de producción, o verificación estática de código— dado que ambos métodos aportan un nivel de confianza distinto y no deben presentarse de forma indistinguible.

Objetivo

Dejar constancia verificable de qué pruebas se realizaron, cómo se realizaron, y qué resultado arrojaron, distinguiendo explícitamente entre validación por ejecución real y validación por revisión estática, para que este documento pueda utilizarse como evidencia auditable del estado de calidad del sistema.

Alcance

Este documento cubre las siete categorías de prueba con evidencia real disponible: pruebas funcionales de backend, pruebas funcionales de frontend, pruebas de integración frontend/backend, pruebas de flujos transaccionales, validación de saldos y movimientos, pruebas de seguridad básica de APIs, y validación de errores y casos borde. **No cubre:** pruebas de integración con "Impact-as-a-Service" (alcance no definido hasta la fecha), validación de interfaces contra maquetas de Figma (no se tuvo acceso al material de diseño durante este trabajo) ni pruebas formales de despliegue separado (se realizaron despliegues independientes del backend de forma iterativa, pero no como ejercicio de prueba formal documentado).

Metodología

Se distinguen explícitamente dos métodos de validación a lo largo de este documento:

- **Ejecución real:** la prueba se realizó enviando peticiones reales (firmadas cuando corresponde) contra los endpoints desplegados en producción, observando la respuesta real del sistema, y —cuando fue necesario— limpiando los datos de prueba generados.
- **Verificación estática:** la prueba consistió en análisis de código (`flutter analyze`), compilación (`flutter build`) o revisión de lógica, sin ejecución interactiva de la interfaz de usuario.

Esta distinción es intencional y se mantiene a lo largo de todo el documento: **el frontend no pudo probarse mediante interacción real de usuario** (no se dispuso de un dispositivo o emulador interactivo durante este trabajo), por lo que su validación se limita a la capa estática. El backend, en cambio, sí se validó mediante ejecución real en múltiples ocasiones.

Desarrollo del Análisis

1. Pruebas Funcionales del Backend

Método: ejecución real.

Se ejecutaron pruebas funcionales directas contra los siguientes componentes, mediante peticiones HTTP reales (firmadas con HMAC donde el diseño lo exige):

Componente probado	Resultado observado
<code>originalauth-register</code>	201 — registro exitoso en Original Auth, <code>original_auth_id</code> capturado
<code>originalauth-verify-email / resend-verification</code>	200 — verificación y reenvío confirmados
<code>originalpay-balance</code>	200 — consulta de saldo real correcta
<code>originalpay-redeem</code>	200 — cobro exitoso con actualización de saldo verificada
<code>originalpay-webhook</code>	200 — recepción y procesamiento de notificación de pago

Todas las pruebas se ejecutaron contra el **entorno de producción real** de cada integración, no contra un entorno simulado.

2. Pruebas Funcionales del Frontend

Método: verificación estática únicamente.

Cada cambio realizado sobre la aplicación Flutter se validó mediante:

- `flutter analyze`: verificación estática de tipos, referencias y reglas de lint sobre la totalidad del código, ejecutada después de cada cambio significativo, con resultado consistentemente limpio (sin errores nuevos, solo advertencias preexistentes no relacionadas).
- `flutter build windows`: compilación completa a un ejecutable real, confirmando que el código no solo pasa análisis estático sino que genera un binario funcional.

Limitación explícita: no se dispuso de un dispositivo físico ni de un emulador interactivo durante este trabajo, por lo que **ningún flujo de interfaz de usuario fue ejercitado mediante interacción real** (toques, navegación, entrada de formularios en vivo). La validación funcional del frontend, en consecuencia, certifica que el código compila y es estáticamente correcto, no que el comportamiento visual coincide con lo esperado.

3. Pruebas de Integración Frontend/Backend

Método: verificación estática del código de integración + ejecución real del lado del backend.

Se verificó, mediante lectura de código, que cada servicio del frontend (`OriginalAuthService`, `AccountService`, `WalletService`, entre otros) invoca correctamente los endpoints correspondientes, con el formato de body y manejo de respuesta esperado. La corrección del lado del backend de esos mismos endpoints se confirmó mediante ejecución real (sección 1). La combinación de ambas evidencias —contrato de cliente correcto + servidor verificado en vivo— constituye la base de esta validación, aunque no equivale a una prueba de integración de extremo a extremo ejecutada mediante interacción real de usuario.

4. Pruebas de Flujos Transaccionales

Método: ejecución real de extremo a extremo.

Se ejecutó el ciclo completo del flujo de mayor complejidad del sistema: cobro con HC vía Original Pay seguido de un rechazo notificado por webhook, verificando:

1. Acreditación de saldo de prueba.
2. Cobro exitoso (`originalpay-redeem`), con descuento confirmado.
3. Notificación de rechazo (`originalpay-webhook`, `payment_status: "rejected"`).
4. Confirmación de que el saldo se restauró exactamente al valor previo al cobro.
5. Reenvío deliberado del mismo webhook, confirmando que el reembolso **no se duplicó** (validación de idempotencia).
6. Limpieza de los datos de prueba generados, restaurando el estado original de la cuenta utilizada.

Este es el flujo transaccional con mayor nivel de evidencia real del proyecto, documentado también en `FLUJOS_TRANSACCIONALES.md`.

5. Validación de Saldos y Movimientos

Método: ejecución real, verificación cruzada.

Se validó, en múltiples ocasiones a lo largo del proyecto, que el saldo (`personal_profiles.hc_balance`) se mantiene consistente con la suma del historial de movimientos (`hc_transactions`), incluyendo un caso donde se detectó una **inconsistencia real**: al eliminar manualmente filas de `hc_transactions` como parte de una limpieza de datos de prueba, el saldo no se ajustó automáticamente (dado que el trigger de sincronización solo reacciona a inserciones, no a eliminaciones). Esta inconsistencia fue identificada,

diagnosticada y corregida mediante un ajuste directo, y quedó documentada como hallazgo en `MODELO_DE_DATOS.md`.

6. Pruebas de Seguridad Básica de APIs

Método: ejecución real.

Se verificó explícitamente el rechazo de peticiones no autorizadas contra los endpoints de integración externa:

- Petición a `originalpay-balance/originalpay-redeem` con firma HMAC inválida → 401.
- Petición a `originalpay-webhook` con credenciales (`token_app/secret_key`) incorrectas → 401.
- Confirmación de que la verificación de JWT a nivel de plataforma de Supabase estaba, en un primer intento, bloqueando incluso peticiones correctamente firmadas por HMAC propio, lo que llevó a la corrección explícita del despliegue (`--no-verify-jwt`) para las funciones destinadas a terceros sin sesión de Supabase.

7. Validación de Errores y Casos Borde

Método: ejecución real.

Se probaron deliberadamente los siguientes casos límite, no solo el camino de éxito:

Caso probado	Resultado observado
Cobro con saldo insuficiente	409 – <code>hc_insuficiente</code>
Consulta/cobro a correo inexistente	404 – <code>usuario_no_encontrado</code>
Registro con formato de país incompatible con Original Auth	400, causa diagnosticada y corregida (se dejó de enviar ese campo)
API key sin aplicación asociada en Original Auth	403, causa diagnosticada y escalada al proveedor
Reintento de webhook ya procesado	Confirmado que no duplica el reembolso
Registro con correo ya existente en Original Auth	409 – El email ya se encuentra registrado, manejado como caso no fatal

Este nivel de prueba de casos borde superó la validación únicamente del camino feliz, y en dos casos (país incompatible, API key sin asociar) permitió identificar y corregir defectos reales antes de que afectaran a un usuario final.

Problemas Detectados

1. **Ausencia total de pruebas automatizadas** (unitarias, de integración o de extremo a extremo) en el repositorio del proyecto. La totalidad de la evidencia documentada en este informe proviene de ejecución manual puntual, no de una suite que pueda re-ejecutarse automáticamente ante cada cambio futuro.
2. **El frontend carece de cualquier evidencia de prueba por interacción real**, dada la limitación de entorno ya explicada. Esto significa que, aunque el código compila y analiza correctamente, no existe confirmación de que el comportamiento visual y de interacción coincida con el esperado.
3. **Las pruebas de integración frontend/backend son parciales**: se verificó la corrección del contrato del lado del cliente y la corrección del backend por separado, pero no se ejecutó una prueba de extremo a extremo real disparada desde una interacción de interfaz.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto alto en sostenibilidad de calidad a mediano plazo**: sin pruebas automatizadas, cada cambio futuro depende de que alguien repita manualmente una validación equivalente a la aquí documentada, con riesgo creciente de omisión conforme el sistema crece.
- El riesgo del punto 2 es de **impacto medio**: el análisis estático limpio reduce, pero no elimina, la probabilidad de defectos de comportamiento visual o de interacción no detectables sin ejecución real.
- El riesgo del punto 3 es de **impacto bajo dado el tamaño actual del proyecto**, pero debe reevaluarse ante flujos de mayor complejidad de interacción entre capas.

Fortalezas Identificadas

- **Evidencia real, no solo teórica, para los flujos financieros más críticos del sistema** (cobro y reembolso con Original Pay), incluyendo verificación explícita de idempotencia.
- **Metodología de prueba que incluyó casos borde y de error de forma deliberada**, no solo el camino de éxito — dos defectos reales fueron detectados y corregidos gracias a esta práctica.
- **Transparencia metodológica**: este mismo documento distingue explícitamente qué se validó por ejecución real y qué por análisis estático, en lugar de presentar ambos niveles de confianza como equivalentes.
- **Limpieza disciplinada de datos de prueba** después de cada validación en el entorno real, evitando dejar residuos de prueba en cuentas de producción.

Áreas de Mejora

- Incorporar una suite de pruebas automatizadas, priorizando los flujos financieros ya validados manualmente en este documento, para convertir esta evidencia puntual en verificación repetible.

- Establecer un mecanismo de prueba de interfaz de usuario (manual estructurada o automatizada) para cubrir la brecha identificada en la sección 2.
- Diseñar al menos un caso de prueba de extremo a extremo real (interacción de usuario → backend → verificación de resultado) para los flujos de mayor criticidad.

Recomendaciones

1. Priorizar la automatización de las pruebas de los flujos ya validados manualmente en este documento (secciones 4, 6 y 7), dado que ya existe el diseño de los casos de prueba — falta únicamente convertirlos en código ejecutable repetible.
2. Programar una sesión de prueba manual estructurada de la interfaz de usuario en un dispositivo o emulador real antes de cualquier entrega o publicación, dado que esta es la brecha de mayor relevancia identificada en este informe.
3. Mantener, como práctica estándar para toda futura integración, el mismo nivel de prueba de casos borde aplicado en las integraciones con Original Auth y Original Pay, dado que demostró valor concreto al detectar defectos reales.

Conclusión

Las pruebas y validaciones realizadas sobre HeartCoin cubren con solidez el lado del backend y las integraciones externas más recientes, con evidencia de ejecución real —no solo teórica— para los flujos financieros de mayor criticidad, incluyendo casos de error y de idempotencia deliberadamente probados. La principal brecha identificada es la ausencia de pruebas automatizadas y de validación por interacción real en el frontend, consecuencia de limitaciones de entorno durante este trabajo más que de una decisión de diseño. Este documento debe leerse como evidencia parcial pero verificable, no como una certificación de cobertura total del sistema.

Anexo: Matriz de Casos de Prueba Ejecutados

Caso	Categoría	Método	Resultado
Registro exitoso en Original Auth	Backend	Ejecución real	201, correcto
Verificación y reenvío de código	Backend	Ejecución real	200, correcto
Consulta de saldo real	Backend / Saldos	Ejecución real	200, correcto

Cobro exitoso con HC	Backend / Transaccional	Ejecución real	200, saldo actualizado correctamente
Reembolso automático por webhook	Transaccional	Ejecución real	Saldo restaurado exactamente
Reintento de webhook (idempotencia)	Transaccional / Errores	Ejecución real	No duplicó el reembolso
Cobro con saldo insuficiente	Errores/casos borde	Ejecución real	409, correcto
Consulta a correo inexistente	Errores/casos borde	Ejecución real	404, correcto
Firma HMAC inválida	Seguridad	Ejecución real	401, correcto
Credenciales de webhook inválidas	Seguridad	Ejecución real	401, correcto
País en formato incompatible	Errores/casos borde	Ejecución real	400 detectado y corregido
API key sin aplicación asociada	Seguridad / Errores	Ejecución real	403 detectado y escalado
Compilación completa del frontend	Frontend	Verificación estática	Exitosa, sin errores
Análisis estático de cada cambio	Frontend	Verificación estática	Limpio en cada iteración